

# UNIVERSIDADE FEDERAL DO PARANÁ

## CI181: Métodos Numéricos – Lista de Exercícios 2

**Professor:** Nicollas Mocelin Sdroievski  
**Alunos:** Gabriel Almeida dos Santos – GRR: 20254589  
Gabriel Gimonski Burger – GRR: 20252011  
Gustavo Lopes – GRR: 20254578  
**Curso:** Engenharia Elétrica  
**Repositório:** <https://github.com/uzumkii/M-todos-Num-ricos>

*Observação: Como escolhemos a modalidade "Programação" para resolver a lista, colocamos aqui as justificativas teóricas curtas e os prints (em formato de texto) do terminal rodando os nossos códigos em Python.*

### Exercício 1 – Eliminação de Gauss-Jordan

---

Montando a matriz aumentada do sistema  $[A|b]$ :

$$\left[ \begin{array}{cccc|c} -2 & -4 & -2 & 2 & -4 \\ -4 & 4 & -4 & 4 & 4 \\ 1 & -1 & 2 & -1 & 2 \\ -1 & 3 & -4 & 4 & -3 \end{array} \right]$$

Aplicando a eliminação de Gauss-Jordan com pivoteamento parcial, o código retornou a matriz diagonal e a solução abaixo:

```
=====
EXERCÍCIO 1 - Eliminação de Gauss-Jordan
=====
Matriz aumentada inicial:
[[-2. -4. -2.  2. -4.]
 [-4.  4. -4.  4.  4.]
 [ 1. -1.  2. -1.  2.]
 [-1.  3. -4.  4. -3.]]

Matriz após Gauss-Jordan (diagonal):
[[-4.  0.  0.  0.  8.]
 [ 0. -6.  0.  0. -6.]
 [ 0.  0. -3.  0. -9.]
 [ 0.  0.  0.  1.  1.]]

Solução (x1, x2, x3, x4): [-2.  1.  3.  1.]
Verificação A @ x == b: True
```

## Exercício 2 – Fatoração LU

Aproveitando a matriz A do primeiro exercício, o código calcula as matrizes L e U. Depois, resolve os sistemas  $Ly = b$  e  $Ux = y$  para achar as soluções para os vetores  $b_1$  e  $b_2$ .

```
=====
EXERCÍCIO 2 - Fatoração LU
=====
Matriz L:
[[ 1.          0.          0.          0.          ]
 [ 2.          1.          0.          0.          ]
 [-0.5        -0.25         1.          0.          ]
 [ 0.5         0.41666667 -3.          1.          ]]

Matriz U:
[[-2.  -4.  -2.   2.]
 [ 0.  12.   0.   0.]
 [ 0.   0.   1.   0.]
 [ 0.   0.   0.   3.]]

Verificação L @ U == A: True

Solução para b1 = [-8.   8.   1.   9.] :
x = [1.  2.  3.  4.]
Verificação A @ x == b1: True

Solução para b2 = [-4.   52.  -16.   44.] :
x = [-1.   5.  -3.   4.]
Verificação A @ x == b2: True
```

## Exercício 3 – Sistema 20x20 (Métodos Diretos)

Para resolver esse sistema maior, usamos a Eliminação de Gauss com pivoteamento parcial no código e, em seguida, a substituição regressiva.

```
=====
EXERCÍCIO 3 - Sistema 20x20 (Gauss com pivoteamento parcial)
=====
Solução x:
x[1] = -4.101222
x[2] = 0.204394
x[3] = -5.500694
x[4] = -1.214729
x[5] = 3.705436
x[6] = 3.570368
x[7] = -1.010808
x[8] = 1.942642
x[9] = -0.747646
x[10] = 4.319147
x[11] = 3.825464
x[12] = -7.127063
x[13] = 1.920200
x[14] = 0.267316
x[15] = 6.102194
x[16] = 0.248812
x[17] = 0.907669
x[18] = 0.280190
x[19] = -3.829828
x[20] = -2.978728
```

```
Verificação A @ x == b: True
```

## Exercício 4 – Gauss-Jacobi (Matriz e Iteração)

O sistema é:

$$\begin{aligned}10x_1 + x_2 &= 11 \\x_1 + 10x_2 + x_3 &= 12 \\x_2 + 10x_3 &= 11\end{aligned}$$

(a) **Forma matricial do método de Gauss-Jacobi:** Isolando as variáveis para chegar na forma iterativa  $x^{(k+1)} = Cx^{(k)} + d$ , a matriz  $C$  e o vetor  $d$  ficam assim:

$$C = \begin{bmatrix} 0 & -1/10 & 0 \\ -1/10 & 0 & -1/10 \\ 0 & -1/10 & 0 \end{bmatrix}, \quad d = \begin{bmatrix} 1,1 \\ 1,2 \\ 1,1 \end{bmatrix}$$

(b) **Garantia de convergência:** Sim, tem garantia. A matriz é estritamente diagonal dominante, porque em todas as linhas o valor absoluto da diagonal principal é maior que a soma dos valores absolutos dos outros elementos da linha:

- Linha 1:  $|10| > |1| + |0|$
- Linha 2:  $|10| > |1| + |1|$
- Linha 3:  $|10| > |0| + |1|$

Isso já é suficiente para garantir que os métodos de Gauss-Jacobi e Gauss-Seidel vão convergir, não importa qual seja o chute inicial escolhido.

## Exercício 5 – Solução Iterativa com $\epsilon = 0.005$

Rodando o método de Gauss-Jacobi começando com  $(0, 0, 0)$ .

```
=====
EXERCÍCIO 5 - Gauss-Jacobi com eps=0.005
=====
Solução com Gauss-Jacobi (erro abs eps=0.005): [0.9996 0.9996 0.9996]
Número de iterações: 4
```

Como o erro absoluto máximo ficou menor que 0.005 logo na iteração 4, o código parou. Arredondando, a solução iterativa converge para  $(1.000, 1.000, 1.000)$ .

## Exercício 6 – Sistema 20x20 Iterativo

Resultados rodando Gauss-Jacobi e Gauss-Seidel pro sistema 20x20, testando o critério de parada tanto por erro absoluto quanto por erro relativo ( $\epsilon = 10^{-6}$ ).

```
=====
EXERCÍCIO 6 - Sistema 20x20 (Jacobi e Seidel)
=====
--- Erro Absoluto (eps = 1e-6) ---
Gauss-Jacobi - iterações: 13
Gauss-Seidel - iterações: 8

--- Erro Relativo (eps = 1e-6) ---
Gauss-Jacobi - iterações: 12
Gauss-Seidel - iterações: 7
```

```
Solução Jacobi (abs): [ 2.34815541 -1.49309832  0.81324864  1.89791646 -0.26792915
1.37424826
2.58940093 -0.14095688  1.78003143  2.22065811 -0.6829676  1.72949926
2.15091177 -1.26063673  1.83688007  0.81735957  1.82555312  0.04017054
1.7812717  2.36634275]
Verificação A @ x == b: True
```

## Exercício 7 – Unicidade do Polinômio Interpolador

---

**Pergunta:** É possível que haja mais de um polinômio interpolador para  $n + 1$  pontos distintos?

**Resposta:** Não, não é possível. Se a gente supor que existem dois polinômios diferentes,  $p(x)$  e  $q(x)$ , ambos com grau  $\leq n$  passando pelos mesmos  $n + 1$  pontos, a diferença entre eles ( $r(x) = p(x) - q(x)$ ) também teria grau  $\leq n$ . Como eles passam pelos mesmos pontos, a função  $r(x)$  teria que ter pelo menos  $n + 1$  raízes (que são os pontos onde  $p$  e  $q$  se encontram). Mas um polinômio de grau  $n$  não pode ter  $n + 1$  raízes, a não ser que ele seja o polinômio nulo ( $r(x) = 0$ ). Ou seja, obrigatoriamente  $p(x) = q(x)$ . Logo, o polinômio é único.

## Exercício 8 – Interpolação de Lagrange

---

Pontos utilizados no código:  $\{(0, 4), (2, 7), (4, 5), (6, 9)\}$ .

```
=====
EXERCÍCIO 8 - Interpolação de Lagrange
=====
p(1) = 6.8125
p(3) = 5.9375
```

## Exercício 9 – Diferenças Divididas e Newton

---

Pontos utilizados no código:  $\{(0, 1), (1, 3), (2, 7), (3, 13), (4, 21)\}$ .

```
=====
EXERCÍCIO 9 - Diferenças divididas e Newton
=====
Tabela de diferenças divididas (F):
[[ 1.  0.  0.  0.  0.]
 [ 3.  2.  0.  0.  0.]
 [ 7.  4.  1.  0.  0.]
 [13.  6.  1.  0.  0.]
 [21.  8.  1.  0.  0.]]

Coeficientes do polinômio de Newton (diagonal de F):
c[0] = F[0,0] = 1.0
c[1] = F[1,1] = 2.0
c[2] = F[2,2] = 1.0
c[3] = F[3,3] = 0.0
c[4] = F[4,4] = 0.0

p(2.5) = 9.75
```

## Exercício 10 – Comparação de Tempo: Lagrange vs Newton

---

```
=====
EXERCÍCIO 10 - Comparação de tempo: Lagrange vs Newton
=====
Tarefa 1 (Lagrange) - Tempo total: 25.2045 segundos
Tarefa 2 (Newton)   - Tempo total: 0.2082 segundos

Lagrange foi 121.1x mais lento que Newton
```

**Justificativa:** Por que deu essa diferença tão grande? Isso acontece porque no método de Lagrange o programa não "guarda" a estrutura do polinômio. Para cada um dos 5000 pontos testados no laço, ele precisa refazer todas as multiplicações das funções de base ( $L_k$ ) do zero, o que dá uma complexidade de avaliação de  $O(n^2)$  para cada ponto.

Já no método de Newton, a gente gasta um tempo  $O(n^2)$  logo no começo pra montar a matriz de diferenças divididas, mas isso é feito **uma vez só**. Depois disso, pra calcular os 5000 pontos, o código usa os coeficientes que já estão prontos e resolve a avaliação em  $O(n)$  (geralmente usando a forma de polinômios aninhados/esquema de Horner), o que deixa a execução dentro do loop muitíssimo mais rápida.